

Inspeção visual de grãos de soja por imagem

Vígil.ia

Documentação técnica do projeto

Versão b1.0.0 (beta) · Demo FATEC · Junho de 2026

Classificação de grãos de soja em 5 classes de qualidade — YOLO11s-cls + OpenCV, com interface web e análise agrônômica conversacional por IA.

Sumário

1. Visão geral do projeto
2. As cinco classes de grão
3. Arquitetura do sistema
4. Pipeline de inferência (modo 1 grão)
5. Stack tecnológico
6. O modelo: migração e desempenho
7. Deploy (Hugging Face Spaces, Vercel, Supabase)
8. Frontend e experiência do usuário
9. Bugs encontrados e correções
10. Fine-tuning v1 — coleta das 500 imagens
11. Histórico de desenvolvimento
12. Estrutura do repositório
13. Roadmap
14. Limitações e observações honestas

1. Visão geral do projeto

A **Vígil.ia** é uma aplicação de **classificação de grãos de soja por imagem**. O usuário fotografa um grão com a câmera do celular e o sistema identifica a classe de qualidade do grão, exibindo o resultado com a confiança do modelo e uma análise agrônômica opcional gerada por IA.

O objetivo imediato é **demonstrativo (demo para a FATEC)**: provar que o modelo reconhece cada classe de grão a partir de uma foto real. Não é, nesta fase, um sistema industrial — esteira, sensor NIR, soprador pneumático e hardware de borda são evolução futura e não fazem parte do escopo atual.

Fluxo, do começo ao fim

foto do celular → redução da imagem → classificação (YOLO11s-cls) → classe + confiança → tela de resultado → conversa opcional com a IA

A marca **Vígil** remete a *vigilância / inspeção*; o sufixo **.ia** marca a camada de inteligência. O produto está em **beta (b1.0.0)**.

2. As cinco classes de grão

O modelo classifica cada grão em uma de cinco classes. Os rótulos internos (em inglês) seguem o dataset; a interface usa os rótulos em português.

0	broken	Quebrado	Roxo
1	immature	Imaturo (verde)	Verde-limão
2	intact	Intacto	Verde
3	skin-damaged	Casca danificada	Âmbar
4	spotted	Manchado	Vermelho

A fronteira mais ambígua do dataset é **skin-damaged × broken** (ambos apresentam dano de superfície) — é onde o modelo mais erra.

3. Arquitetura do sistema

O sistema tem três peças independentes. O modelo de visão (torch/ultralytics) não cabe no limite serverless da Vercel, por isso a visão roda em um serviço separado.

Frontend + LLM	Vercel (Next.js)	Interface, tela de resultado e a rota /api/explain (chat com a IA via Groq). Não usa Python.
Visão (/inspect)	Hugging Face Spaces (Docker)	FastAPI + YOLO11s-cls + OpenCV. O modelo .pt embutido na imagem.
Banco de dados	Supabase	Tabelas inspecoes e lotes; persiste o resultado de cada inspeção.

Fluxo entre as peças

Navegador → (foto) → HF Space /inspect (YOLO) → resultado + grava no Supabase
Navegador → (pergunta) → Vercel /api/explain → Groq · Llama 3.3 (streaming)

4. Pipeline de inferência (modo 1 grão)

A versão atual opera em **modo 1 grão**: a foto inteira é classificada como um único grão. O modo multi-grão (segmentação OpenCV de vários grãos por foto) existe no código e é acionável por uma flag, ficando reservado para a fase de lote/esteira.

- **Captura**: o usuário envia/fotografa um grão; a imagem é lida uma única vez.
- **Redução**: a imagem é reduzida para no máx. 1280 px (JPEG) no navegador — cabe no armazenamento local e alivia o upload, sem afetar a classificação (o modelo usa 224 px).
- **Classificação**: o backend recebe a imagem, o YOLO11s-cls prediz a classe e a confiança.
- **Mapeamento**: os nomes do modelo (ex.: 'Broken soybeans') são normalizados para os rótulos internos ('broken').
- **Resposta**: total de grãos, contagem por classe, caixa(s) e dimensões da imagem; o registro é gravado no Supabase.
- **Análise opcional**: na tela de resultado, o usuário pode conversar com a Vígil.ia sobre a classe identificada (IA não é chamada até que o usuário pergunte).

5. Stack tecnológico

Treino do modelo	Ultralytics / PyTorch (Google Colab, GPU grátis)
Modelo	YOLO11s-cls (classificação) via transfer learning
Segmentação (recorte)	OpenCV clássico (grayscale → Otsu → maior contorno → bbox)
Backend de visão	FastAPI + Uvicorn (Docker no Hugging Face Spaces)
LLM (análise)	Groq · Llama 3.3 70B (API compatível com OpenAI)
Frontend	Next.js 14 + TypeScript + Tailwind CSS (Vercel)
Banco de dados	Supabase (PostgreSQL gerenciado)
Dataset	Roboflow 'SoyaBeans Classifications v2' — MIT, ~12.528 imagens, 400×400

Pinos relevantes do backend: torch==2.2.2, torchvision==0.17.2, ultralytics==8.3.0 — escolhidos para carregar o modelo YOLO11 sem erros (ver seção 9).

6. O modelo: migração e desempenho

O classificador de produção foi **migrado de EfficientNet-B0 (TensorFlow/Keras) para YOLO11s-cls (Ultralytics/PyTorch)** após um experimento controlado nas mesmas 57 fotos reais, mesmo recorte, mesmo split e augmentation equivalente. Mede-se a acurácia no **domínio real** (fotos de celular, fundo cinza).

EfficientNet-B0 — receita antiga	64,0%	25,0% (overfitting)
EfficientNet-B0 — em igualdade de condições	75,0%	7,2%
YOLO11s-cls — produção	91,7%	3,9%

Por que o YOLO generaliza melhor com poucas fotos

- **AdamW + weight decay**: regularização no otimizador (vs Adam puro do EfficientNet).
- **EMA** (média móvel dos pesos): suaviza o checkpoint final e melhora a generalização.
- **Atenção espacial (C2PSA)**: o último bloco do YOLO11s foca na textura/cor do grão e ignora o fundo — exatamente o problema de domain shift.

O gargalo real: domain shift

O modelo treina com fotos de **fundo preto** (dataset Roboflow); a foto do celular tem **fundo cinza/metálico** e luz variada. Sem adaptação, a acurácia despenca para **~3-8%**. O **fine-tuning no domínio real** (freeze parcial + augmentation forte de brilho/cor) recupera para 91,7%. Por isso o fine-tuning deixou de ser opcional.

Honestidade metodológica

A validação real tinha apenas **12 fotos** — 91,7% equivale a 11/12. É uma tendência forte, não uma métrica estatística precisa. Por isso a próxima etapa é coletar 500 imagens (seção 10) para um val/test sólidos.

7. Deploy

Backend de visão — Hugging Face Spaces

- Space Docker Guguinhaxd/soja-inspection-api (URL: guguinhaxd-soja-inspection-api.hf.space).
- Dockerfile com usuário uid 1000, caches graváveis e porta 7860; modelo .pt embutido na imagem.
- Secrets do Space: SUPABASE_URL e SUPABASE_KEY (service_role).
- Healthcheck: GET /health → {"status":"ok"}.

Frontend — Vercel

Import pelo painel com **Root Directory = frontend** e Framework Preset = Next.js. Variáveis de ambiente:

NEXT_PUBLIC_SUPABASE_URL	URL do Supabase (browser)
NEXT_PUBLIC_SUPABASE_ANON_KEY	Chave anon pública (browser)
NEXT_PUBLIC_API_URL	URL do HF Space (rota /inspect)
GROQ_API_KEY	Chave da Groq (server-side, sem prefixo NEXT_PUBLIC_)

Banco — Supabase

Tabelas inspecoes (resultado de cada inspeção) e lotes, com RLS habilitado e políticas de leitura/inserção pública. Recomenda-se rotacionar as chaves após o deploy e revisar o RLS antes de uso público.

8. Frontend e experiência do usuário

Identidade visual **preta e sofisticada**, inspirada na sobriedade do app do Claude, com um verde de marca como acento e um símbolo animado próprio.

Principais elementos

- **Símbolo (InspectionLogo)**: uma íris/abertura de precisão (pupila, anéis, bezel de micro-ticks e palitinhos radiais) com um arco de varredura. Ao inspecionar, os palitinhos giram rápido e a íris devagar, em sentido horário — sensação de varredura.
- **Abertura (IntroSplash)**: ao entrar no site, o símbolo se desenha e a marca 'Vígil' aparece; depois o splash some e revela a tela inicial. Toca uma vez por sessão.
- **Tela inicial (hero)**: símbolo central + 'Inspeccionar', aceitando clique ou arrastar.
- **Tela de resultado**: card de veredito com anel de confiança animado na cor da classe, selo de status (Íntegro × Defeito), tabela com barras de porcentagem, moldura da imagem com brilho na cor da classe e animações de entrada escalonadas.
- **Chat com a IA**: barra de digitação com respostas em **streaming** (token a token), histórico contextual, indicador 'digitando', botão limpar e **modo tela cheia**.
- **Aba Sobre**: explica o que é a Vígil.ia, o desempenho e a linha do tempo das atualizações.
- **Acessibilidade**: todas as animações respeitam `prefers-reduced-motion`.

9. Bugs encontrados e correções

A integração ponta a ponta revelou uma série de problemas reais, corrigidos um a um:

Space não subia (UnpicklingError)	torch ≥ 2.6 usa <code>weights_only=True</code> ; ultralytics 8.2.0 não allowlista os globals	Fixar <code>torch==2.2.2</code> / <code>torchvision==0.17.2</code>
Erro 'Can't get attribute C3k2'	O .pt é um YOLO11 (bloco C3k2), inexistente no ultralytics 8.2.0	Subir ultralytics para 8.3.0
ValueError 'Broken soybeans'	O modelo usa rótulos longos; o código comparava com os curtos	Normalizar os nomes de classe
Upload dava erro mesmo com HTTP 200	CORS com wildcard '*' + <code>allow_credentials=True</code> (inválido pelo spec)	<code>allow_credentials=False</code>
'Não consegui ler o arquivo'	A mesma foto era lida duas vezes; a 2ª falhava (Safari/iOS)	Ler o arquivo uma única vez e reusar
Quota do sessionStorage estourava	Data URL da foto do celular tem vários MB	Reduzir a imagem antes de guardar/enviar
Site travava na abertura	sessionStorage lança no Safari/privado e o splash não terminava	Blindar com try/catch + memória de sessão
Tela cheia do chat 'quebrada' no celular	Faltava <code>min-h-0</code> ; a barra de digitação era empurrada pra fora	<code>min-h-0</code> + altura 100dvh + área segura

10. Fine-tuning v1 — coleta das 500 imagens

Para reduzir o domain shift e validar com solidez, a meta é coletar **~100 fotos por classe (500 no total)**, com captura manual, e usar o notebook `model/finetune_yolo.ipynb`, adaptado para ler do **Google Drive**.

Protocolo de captura

- 1 grão por foto, centralizado e em foco (o notebook recorta o maior contorno via Otsu).

- Mesmo setup da demo: o mesmo celular, fundo e luz que serão usados na apresentação.
- Variar de propósito dentro de cada classe (ângulo, iluminação, distância).
- Rotular corretamente: cada grão na pasta da sua classe verdadeira.
- Evitar: grãos se tocando, sombra dura, borrão, dois grãos no quadro.
- Atenção especial em skin-damaged × broken (a fronteira mais difícil).

Pipeline do notebook (FT-0 a FT-5)

- FT-0 bootstrap: carrega o modelo base do Drive e define o recorte.
- FT-1: lê do Drive e faz split estratificado 70/15/15 (train/val/test).
- FT-2: baseline do modelo antes do fine-tuning.
- FT-3: fine-tuning (freeze parcial, augmentation forte, batch 32, 60 épocas).
- FT-4: antes × depois em val e test + matriz de confusão.
- FT-5: salva o modelo apenas se o val melhorar (gate de segurança), versionado.

11. Histórico de desenvolvimento

Resumo cronológico do trabalho (mais recente primeiro), agrupado por tema:

Modelo e migração

- Migração de EfficientNet/Keras para YOLO11s-cls; experimento justo comparando as arquiteturas.
- Notebook de fine-tuning no domínio real (backbone congelado, augmentation forte).
- Adaptação do notebook para ler do Drive e escalar para 500 (split train/val/test).

Deploy e backend

- Adaptação do backend de visão para Hugging Face Spaces (Docker) e publicação do Space.
- Correções de carga do modelo: torch<2.6, ultralytics 8.3.0, normalização de classes.
- Modo 1 grão por padrão (multi-grão preservado via flag).

Frontend e experiência

- Tema preto + símbolo animado; marca Vígil.ia (b1.0.0 beta) e animação de abertura.
- Tela de resultado legível e agradável: veredito, anel de confiança, barras, status.
- IA opcional → chat com barra de digitação → streaming + tela cheia + limpar conversa.
- Remoção da Dashboard e criação da aba 'Sobre Vígil.ia'.

Estabilização (bugs)

- CORS válido, leitura única do arquivo, redução de imagem (quota), erro de inspeção legível.
- Abertura sem flash e à prova de falha de storage; tela cheia do chat corrigida no celular.

12. Estrutura do repositório

```
frontend/ App Next.js (UI, /api/explain, componentes)
  src/app/ páginas: home, resultado/[id], sobre, api/explain
  src/components/ InspectionLogo, IntroSplash, InspectHero, ResultVerdict,
                  DefectTable, BoundingBoxCanvas, ExplainPanel
  src/lib/ api.ts, supabase.ts
backend/ FastAPI (main.py, inference.py, database.py, Dockerfile)
  soja_yolo11s_finetuned.pt modelo embutido
model/ notebooks de treino e fine-tuning, classes, scripts
supabase/migrations/ schema inicial (inspecoes, lotes)
docs/ especificação e materiais de apoio
```

13. Roadmap

- Coletar 500 correções/fotos e treinar o fine-tuning v1.
- Reativar o modo multi-grão (inspeção de lote inteiro) via segmentação OpenCV.
- Router de domínio leve para despachar grãos × superfícies sob uma API única.
- Persistência das imagens e fluxo de correção humana para crescer o dataset.
- Evolução futura (fora do escopo da demo): esteira, NIR, ejeção pneumática, hardware de borda.

14. Limitações e observações honestas

- Validação real ainda pequena (12 fotos) — 91,7% é tendência, não métrica estatística.
- Domain shift é o gargalo: a captura precisa imitar o setup do dataset/uso real.
- O Hugging Face Spaces (free) hiberna após inatividade; a 1ª chamada tem cold start.
- skin-damaged × broken é a confusão mais provável do modelo.
- Segurança: revisar RLS e rotacionar chaves antes de qualquer uso público.

Vígil.ia · b1.0.0 (beta) · Documentação gerada em junho de 2026. Projeto de classificação de grãos de soja — demo acadêmica (FATEC).